

Edge Detection

Edge Detection

Preface

Example Results

Principle Explanation

First and Second Order Derivatives

Code Overview

Initialize Camera and Display Module

Set Detection Mode

Convert to Grayscale

Execute Edge or Gradient Computation

Package Single-Channel Result and Display

Resource Release

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the jianan website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/

选择开发板

CanMV K230 V1.0/V1.1

CanMV K230 V3.0

庐山派 K230

庐山派 Lite K230D

Yahboom K230

01Studio K230

01Studio K230 Emmc

Hiwonder K230

WonderMK K230

GT6700 K230

勘智

KENDRYTE

Yahboom K230

MicroPython Build for Yahboom K230

当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-20-g9b151af_nncase_v2.11.0.img.gz](#)

最新每日构建版本

Daily-0728

编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa

立即下载

极简 USB 烧录指南 (推荐)

⚠ 镜像仅可烧录到对应的开发板, 请务必选择与当前板型完全匹配的镜像, 不能混烧其他板型的镜像。否则可能导致硬件损坏!!!

1 准备工具

下载并打开 [K230BurningTool](#) 烧录工具, 并加载固件。

2 进入烧录模式 (根据硬件情况选择)

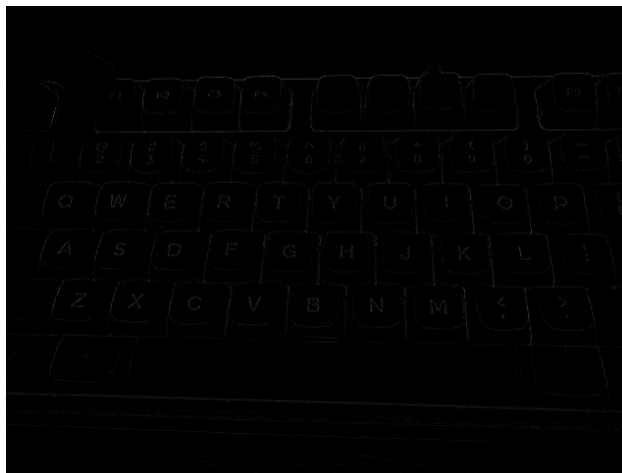
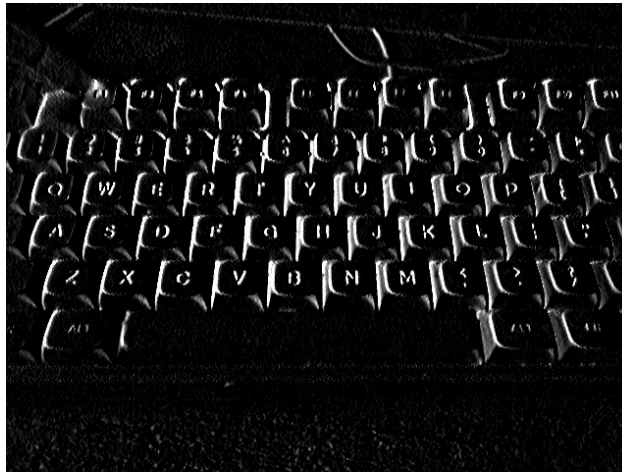
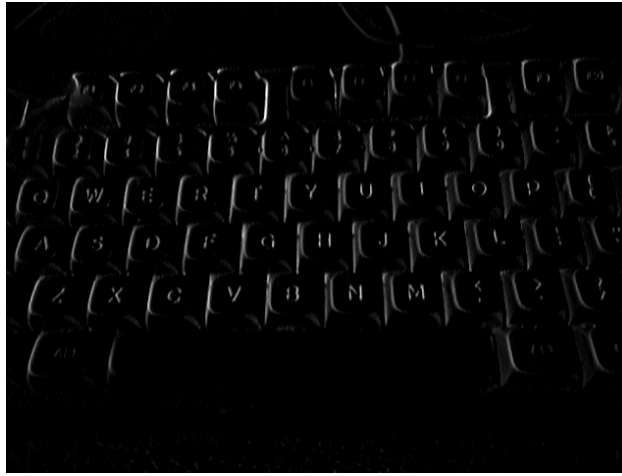
有 BOOT 键的板子

无 BOOT 键的板子

Example Results

Run the example code for this section [11.camera_edge_detect.py](#).

This section first converts the live color image to grayscale, then demonstrates Canny, Sobel, Scharr, and Laplacian edge or gradient operators. The output is displayed as a single-channel grayscale image.



Principle Explanation

First and Second Order Derivatives

Image edges correspond to locations where grayscale changes rapidly. Sobel and Scharr approximate first-order derivatives, detecting gradients in the x or y direction respectively. Laplacian computes second-order derivatives, which is more sensitive to details and noise.

Canny is a multi-stage algorithm that typically includes smoothing, gradient computation, non-maximum suppression, and double threshold linking, outputting relatively thin and continuous binary edges.

Algorithm	Direction in Example	Feature
Canny	All directions	Double threshold linking, thin edges
Sobel	x direction	General first-order gradient
Scharr	x direction	Better rotational symmetry with small kernels
Laplacian	All directions	Second-order derivative, sensitive to noise

Code Overview

Initialize Camera and Display Module

```
w, h = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=w, height=h)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Set Detection Mode

```
mode = 0
```

\$mode\$	Algorithm	Source Parameters
0	Canny	Low threshold 80, high threshold 200, source default
1	Sobel	\$CV_8U\$, dx=1, dy=0
2	Scharr	\$CV_8U\$, dx=1, dy=0
3	Laplacian	\$CV_8U\$

Convert to Grayscale

```
img = sensor.snapshot()
img_np = img.to_numpy_ref()
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
```

Edge detection usually does not need color information. Grayscale reduces data volume and unifies channel processing.

Execute Edge or Gradient Computation

```
if mode == 0:
    out = cv2.Canny(gray, 80, 200)
elif mode == 1:
    out = cv2.Sobel(gray, cv2.CV_8U, 1, 0)
elif mode == 2:
    out = cv2.Scharr(gray, cv2.CV_8U, 1, 0)
else:
    out = cv2.Laplacian(gray, cv2.CV_8U)
```

The current example directly outputs CV_8U. Negative gradients cannot be fully preserved in unsigned 8-bit output, so results are suitable for visual demonstration but not for direct precise gradient numerical analysis.

Package Single-Channel Result and Display

```
img_out = image.Image(
    W, H, image.GRAYSCALE,
    alloc=image.ALLOC_REF,
    data=out
)
Display.show_image(img_out)
```

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Canny Double Threshold:** The high threshold determines strong edges, and the low threshold is used to connect weak edges adjacent to strong edges. The low threshold is usually less than the high threshold.
- **Pre-denoising:** The current script does not have Gaussian preprocessing. When there are many noise edges, you can first add 3×3 or 5×5 Gaussian blur.
- **Gradient Direction:** Sobel/Scharr with dx=1, dy=0 computes changes in the x direction, highlighting edges close to vertical. Changing to dx=0, dy=1 allows observation of the other direction.
- **Output Depth:** When you need to preserve positive and negative gradients, use firmware-supported signed or floating-point depths, then take the absolute value and normalize for display.
- **Laplacian Noise:** Second-order derivatives amplify high-frequency noise, usually relying more on pre-smoothing.
- **Performance:** You can downsample when only doing detection. When you need to draw edge coordinates back onto the original image, you must synchronously scale the coordinates.